

Contents

Cortex: AI as PID 1	3
Abstract	3
1. Introduction	3
1.1 The Problem	3
1.2 The Thesis	4
1.3 The Agent Microkernel	4
2. Architecture	4
2.1 Tier System	4
2.2 The Core Ladder	5
2.3 Infinity Models: Dynamic Discovery	6
2.4 GGUF-First Format Strategy	6
3. Routing	7
3.1 The L0 Router	7
3.2 Complexity Estimation	7
3.3 Tier Clamping	7
4. Confidence Verification	7
4.1 The Problem with Single-Model Inference	7
4.2 Cross-Family Challenge	8
4.3 Challenge Model Sources	8
5. Swarm Consensus	9
5.1 When Challenge Fails	9
5.2 Clustering and Voting	9
5.3 Cross-Family Signal	9
6. Escalation Pipeline	10
7. Tool Registry	11
7.1 The Device Driver Model	11
7.2 Permission Rings	11
7.3 Built-in Tools	11
8. Policy Engine	12
8.1 Hierarchical Scoping	12
8.2 Policy Types	12
8.3 Enforcement	12
9. Resilience	13
9.1 Circuit Breakers	13
9.2 Retry with Backoff	13
9.3 Tier Waterfall	13
10. Persistent Memory	14
10.1 Storage Layer	14
10.2 Context Management	14
11. System Layer	14
11.1 Hardware Detection	14
11.2 Model Manager	14
11.3 Backend Adapters	15
12. API Surface	15
12.1 The Daemon	15
12.2 Multi-Format Support	15

12.3	Kernel Endpoints	15
12.4	Compatible Clients	16
13.	Semantic Compression Language (SCL)	16
13.1	The Control Protocol	16
13.2	Grammar	16
13.3	Core Types	16
13.4	Examples	17
13.5	Semantic State Deltas	17
13.6	Applications	18
13.7	Gossip Protocol	18
13.8	Executable SCL — The Evaluator	20
14.	Braille Encoding Layer	21
14.1	The Codec	21
14.2	Fingerprinting	22
14.3	Manifests	22
14.4	The Semantic-First Substrate	23
15.	Self-Training: The Cortex Kernel Model (CKM)	24
15.1	Motivation	24
15.2	Architecture	24
15.3	Braille-Native Tokenization	24
15.4	Inference Without Dependencies	25
15.5	Training Pipeline	25
15.6	Safety Policy	25
15.7	Two Planned Specializations	26
16.	Self-Modification	26
16.1	The Observe-Mutate Loop	26
16.2	Policy Rewriter	26
16.3	Boot Self-Modification	27
16.4	Convergence with Research	27
17.	Acoustic Gossip Transport	27
17.1	Motivation	27
17.2	Encoding: Binary FSK	27
17.3	Transmission Times	28
17.4	Demodulation	28
17.5	The Full Stack	28
17.6	Zero Dependencies	29
18.	Performance Characteristics	29
18.1	TTFT Optimization	29
18.2	Platform-Specific Performance	29
18.3	Graceful Degradation	29
18.4	L7: From API to Collective	30
19.	Design Principles	31
20.	Future Work	31
21.	Conclusion	32

Cortex: AI as PID 1

An agent microkernel where the inference engine isn't an app — it's `init`.

Preprint. June 2026. Not peer-reviewed. Ryan Barrett — Elevate Foundry Repository:
github.com/elevate-foundry/cortex

Abstract

Modern AI-assisted computing treats large language models as user-space applications: launched on demand, isolated from the system, and discarded after use. This is analogous to the pre-Unix era, where every program had to implement its own I/O routines. Cortex proposes a different architecture: the LLM inference engine as PID 1 — the always-running, kernel-adjacent process that mediates between the user and everything else.

Cortex detects available hardware, constructs a tiered model hierarchy (L0–L7), and routes every request to the smallest model capable of handling it. When confidence is low, it escalates — first to a cross-family challenger model, then to a multi-model swarm, and finally to a remote frontier API. Any model the user installs — regardless of family, quantization, or size — is automatically discovered, tier-classified, and made available as a challenger or swarm participant. The result is an inference layer that behaves like an operating system scheduler: resource-aware, always available, self-correcting, and model-agnostic.

Beyond routing, Cortex implements the full agent microkernel: a tool registry with permission rings, a policy engine for per-app access control, a resilient execution layer with circuit breakers and fallback, persistent memory for conversations and audit trails, and Semantic Compression Language (SCL) as its native control protocol. Every request is a `syscall`. Every model is a device driver. Every tool is a capability. Every policy is a permission boundary.

1. Introduction

1.1 The Problem

Today's LLM deployment follows a client-server model inherited from web applications. A user opens an app, the app calls an API, a large model processes the request, and the response returns. This architecture has four structural inefficiencies:

1. **Overprovisioning.** Most requests are routed to the largest available model regardless of complexity. A `yes/no` classification consumes the same resources as a multi-step coding task.
2. **Cold starts.** Models are loaded on demand and evicted under memory pressure. The resulting latency — often seconds — makes LLMs unsuitable as system-level services.
3. **Single points of failure.** A request goes to one model. If that model hallucinates, there is no second opinion. Confidence is assumed, never measured.
4. **No permission model.** Applications and tools operate with unconstrained access. There is no kernel-level enforcement of what a model can do, what data it can access, or what actions it can take.

1.2 The Thesis

An operating system doesn't ask which process scheduler to start. It doesn't load the memory manager on demand. These are kernel services — always running, always available, mediating access to hardware on behalf of user-space programs.

Cortex applies this principle to inference. The L0 router is PID 1: the first process up, the last one down. It classifies every inbound request, selects the appropriate tier, and supervises execution. Tiers L0–L2 remain resident in memory like kernel modules. Higher tiers load on demand like user-space daemons. Remote frontier models (L7) are the network — a fallback when local resources are insufficient.

1.3 The Agent Microkernel

Beyond the inference scheduler, Cortex implements six kernel subsystems:

Subsystem	OS Analogy	Module
Tier Routing	Process scheduler	<code>router.py</code>
Model Manager	<code>systemd</code>	<code>model_manager.py</code>
Persistent Memory	Filesystem + <code>/proc</code>	<code>memory.py</code>
Tool Registry	Device driver interface + capabilities	<code>tools.py</code>
Policy Engine	SELinux / AppArmor	<code>policy.py</code>
Resilience Layer	Watchdog + circuit breakers	<code>resilience.py</code>

Together these form a complete agent microkernel: any application can make inference syscalls, invoke tools, and operate within enforced permission boundaries — all through a single localhost endpoint.

2. Architecture

2.1 Tier System

Cortex organizes models into eight tiers mapped to operating system concepts:

Tier	Parameters	OS Analogy	Role	Resident	TTFT Target
L0	0.5–1B	<code>init</code> / <code>systemd</code>	Router, classifier, reflex	Always	~10ms
L1	1–2B	Syscall interface	Tool calls, summarization	Always	~20ms

Tier	Parameters	OS Analogy	Role	Resident	TTFT Target
L2	3–4B	Shell / window manager	File ops, drafts, structured extraction	Always	~40ms
L3	7–8B	User-space daemon	Coding, multi-step tasks	On demand	~60ms
L4	12–14B	Heavy daemon	Debugging, planning, complex reasoning	On demand	~100ms
L5	30–32B	Workstation service	Strong local reasoning	On demand	~200ms
L6	64–70B	Full workstation	Frontier-adjacent local quality	On demand	~400ms
L7	Frontier	Network / remote API	Escalation when local fails	On demand	~500ms

Always-resident tiers (L0–L2) stay loaded in VRAM at all times. Their combined footprint is approximately 4.5 GB at 4-bit quantization — small enough to coexist with higher tiers on any modern GPU or Apple Silicon machine. They never swap out, ensuring sub-50ms time-to-first-token for the majority of requests.

On-demand tiers (L3–L6) load when needed and are evicted under memory pressure, managed by the Model Manager in the same way `systemd` manages service lifecycles.

L7 is not a local model. It is a passthrough to remote frontier APIs (OpenAI, Anthropic) — the “network” in the OS analogy. It activates only when local models cannot reach sufficient confidence.

2.2 The Core Ladder

Cortex uses Qwen3 as its primary model family across all local tiers:

	Tier	Model
	L0	Qwen3-0.6B
	L1	Qwen3-1.7B

	Tier	Model
L2		Qwen3-4B
L3		Qwen3-8B
L4		Qwen3-14B
L5		Qwen3-30B-A3B / Qwen3-32B
L6		Qwen2.5-72B

A single model family provides a consistent “core ladder” — predictable behavior, shared tokenization, and uniform instruction formatting. The choice of Qwen3 is pragmatic: it offers the widest range of sizes (0.6B to 32B) from a single architecture, with strong multilingual performance and native tool-calling support.

2.3 Infinity Models: Dynamic Discovery

The core ladder is the *default*. It is not the limit. Cortex dynamically discovers every model installed in Ollama (or any compatible backend) and automatically classifies it:

1. **Parameter-based tier assignment.** A model’s parameter count maps it to a tier: Phi-4 (14B) → L4, Llama 3.3 (70B) → L6, DeepSeek-R1 (7B) → L3.
2. **Family extraction.** The model’s training family (llama, gemma, granite, phi, deepseek, mistral, olmo, smollm, etc.) is normalized and tracked.
3. **Automatic challenge enrollment.** Any discovered model from a *non-core* family at any tier becomes available as a challenger. You don’t curate challengers — you pull models and they become challengers.
4. **Swarm pool expansion.** The swarm draws from all available models at a tier: core + curated challengers + discovered models. More models installed = wider cross-family coverage = higher confidence ceilings.

The model census is queryable at runtime via `GET /v1/models/census` and refreshable via `POST /v1/models/discover`. A system with 50 Ollama models has 50 potential participants in confidence verification — the challenge and swarm pools scale to infinity.

```
@cortex → discover [source: ollama, auto_classify: true, auto_enroll: true]
@discovery → result [models: 47, families: 9, tiers_covered: L0-L6]
@phi4:14b → classify [tier: L4, family: phi, role: challenger]
@deepseek-r1:7b → classify [tier: L3, family: deepseek, role: challenger]
@llama3.3:70b → classify [tier: L6, family: llama, role: challenger]
```

2.4 GGUF-First Format Strategy

Cortex standardizes on GGUF (llama.cpp’s quantized model format) as the universal model file format. A single GGUF file runs identically on:

- **NVIDIA CUDA** (Linux, Windows)
- **AMD ROCm** (Linux)
- **Apple Metal** (macOS)
- **CPU** (AVX2, NEON — any platform)

This eliminates the format fragmentation problem. There is no need for separate model files per platform. For NVIDIA users running vLLM, Cortex optionally uses AWQ-quantized models for better prefix caching performance, but falls back to GGUF universally.

3. Routing

3.1 The L0 Router

Every request passes through L0 before any work begins. The router performs two functions:

1. **Task classification.** Categorizes the request into one of ten categories: `classify`, `tool_call`, `multi_tool`, `code`, `debug`, `plan`, `analyze`, `generate`, `safety`, or `unknown`.
2. **Tier selection.** Maps the category and estimated complexity to the minimum viable tier.

When the L0 model (Qwen3-0.6B) is loaded, routing uses the model itself — a 0.6B classifier that reads the prompt and outputs a JSON object specifying tier, category, and confidence. When no L0 model is available, Cortex falls back to rule-based heuristics using regex pattern matching and complexity scoring.

3.2 Complexity Estimation

The heuristic router estimates complexity on a 0–1 scale using:

- **Input length** — longer prompts generally require larger context windows
- **Pattern matching** — code keywords, planning language, tool verbs, safety indicators
- **Multi-step indicators** — conjunctions like “then”, “after that”, “next”
- **Tool count** — number of distinct tool-call verbs in the prompt

Each category also has a minimum tier floor. For example, `code` requires at least L3; `safety` requires at least L4. The router takes the maximum of the category floor and the complexity-derived tier.

3.3 Tier Clamping

If the selected tier exceeds what the hardware can run, the router clamps to the maximum feasible tier and attaches an **escalation hint** pointing to L7. Confidence is reduced proportionally to the gap between the ideal and actual tier. The policy engine may further clamp the tier: an app-specific `max_tier`: L2 policy constrains that application to L0–L2 regardless of task complexity.

4. Confidence Verification

4.1 The Problem with Single-Model Inference

A single model’s output has no external check. It may be fluent but wrong. Traditional approaches address this with temperature sampling or self-consistency (querying the same model multiple times), but these offer limited value: copies of the same model share the same blind spots.

4.2 Cross-Family Challenge

Cortex maintains a challenge model pool — models from families other than the core ladder (Llama, Gemma, Granite, Phi, OLMo, SmolLM, DeepSeek, Mistral, and any other discovered family). These models were trained on different data, with different architectures and different optimization objectives. When they agree with the core model, the probability of a shared hallucination is substantially lower than agreement among copies of the same family.

The challenge process:

1. The core model (Qwen) generates an answer.
2. If routing confidence is below 0.75, the Challenger selects a model from a different family at the same tier.
3. The challenger model answers the same prompt independently.
4. Answers are compared using normalized text overlap, yes/no detection, and conclusion extraction.
5. Agreement is classified as: **strong_agree**, **weak_agree**, **ambiguous**, **disagree**, or **strong_disagree**.

Agreement	Confidence	Action
	Strong agree	0.85–0.95
	Weak agree	0.65
	Ambiguous	0.45
	Disagree	0.25
	Strong disagree	0.10

4.3 Challenge Model Sources

Challenge models come from two sources:

1. **Curated catalog.** Known-good models per tier, tested for compatibility:

Tier	Curated Challenge Families
L0	Llama 3.2, OLMo 2, Gemma 3
L1	Llama 3.2, Granite 3.3, SmolLM3
L2	Gemma 3, Phi-4 mini
L3	Granite 3.3, Llama 3.1
L4	Gemma 3, Phi-4
L5	Granite 3.3, OLMo 2
L6	Llama 3.3

2. **Discovered models.** Any model installed in Ollama from a non-Qwen family is automatically added to the challenge pool at its corresponding tier. Running `ollama pull deepseek-r1:7b` instantly adds a DeepSeek challenger at L3.

5. Swarm Consensus

5.1 When Challenge Fails

If the challenger disagrees with the core model — or if initial confidence is very low — Cortex escalates to a **swarm**: a parallel fan-out to multiple models across multiple families.

Difficulty	Swarm Size	Strategy
Hard	3–5 models	Small swarm, weighted vote
Hardest	All available	Large swarm, weighted consensus

With infinity models, the “all available” swarm can include every installed model at a tier — core, curated challengers, and discovered models. A machine with 8 models at L3 produces an 8-way swarm with potentially 5+ independent families.

5.2 Clustering and Voting

Swarm responses are grouped into **agreement clusters** using pairwise comparison (the same text-overlap algorithm used by the Challenger). Two responses land in the same cluster if they exhibit strong or weak agreement.

Weighted voting determines the winner. Each model’s vote is weighted by its parameter count:

Parameters	Weight
1B	1.0
4B	1.5
8B	2.0
14B	2.5
32B	3.0
70B	4.0

The winning cluster’s highest-weighted model provides the final answer. Confidence is computed as:

$$\text{confidence} = \text{agreement_ratio} + \text{family_bonus} - \text{size_penalty}$$

Where: - **agreement_ratio** = fraction of total weight in the winning cluster - **family_bonus** = 0.05 per agreeing family (capped at 0.15) - **size_penalty** = penalty for small swarms (diminishes with more models)

5.3 Cross-Family Signal

The key insight is that **cross-family agreement is a stronger signal than within-family agreement**. Three models from three different families converging on the same answer provides higher confidence than five copies of the same family agreeing — because independent training pipelines, data mixtures, and architectures make correlated failures less likely. The infinity model architecture maximizes this signal by automatically enrolling every installed model into the cross-family verification pipeline.

6. Escalation Pipeline

The full processing pipeline follows a deterministic escalation path:

User Request

Policy Check Rate limit, permissions, tier cap

denied? → 403 Forbidden (reason)

Router L0 classifies → selects tier
(L0)

Core Model Qwen at selected tier generates answer
(Qwen3)

confidence < 0.75?

No Yes

Return

Challenger Different family verifies (curated + discovered)

agree?

Yes No

Return

Swarm All available models, weighted consensus

consensus?

Yes No

Return Escalate → L7 (remote frontier)

Each step is logged to the audit trail with the full escalation path. The resilience layer wraps each

backend call with circuit breakers and retry logic; if a backend fails, the system automatically falls back to the next available model.

7. Tool Registry

7.1 The Device Driver Model

In a traditional OS, device drivers provide a controlled interface to hardware. Applications don't access the disk directly — they call `read()` and `write()` through the kernel, which enforces permissions.

Cortex applies the same principle to tool calling. When a model invokes a tool (function calling), the request passes through the Tool Registry, which:

1. **Validates** the tool exists and is enabled
2. **Checks permissions** against the privilege ring
3. **Enforces rate limits** per tool
4. **Executes** the tool in a controlled context
5. **Audits** every invocation

7.2 Permission Rings

Tools are classified into five privilege rings, from safest to most dangerous:

Ring	Level	Description	Examples
0	READ_ONLY	No side effects. Info retrieval only.	<code>web_search</code> , <code>get_time</code> , <code>read_file</code>
1	DRAFT	Creates artifacts, doesn't execute them.	<code>code_interpreter</code> (sandbox), staged file writes
2	EXECUTE	Local side effects. Modifies local state.	<code>shell</code> , <code>write_file</code> , <code>git_commit</code>
3	EXTERNAL	External side effects. Network mutations.	<code>send_email</code> , <code>post_to_api</code> , <code>deploy</code>
4	DANGEROUS	Destructive / irreversible.	<code>delete_database</code> , <code>rm -rf</code> , <code>transfer_funds</code>

Each request context has a `max_ring` (set by the policy engine). A tool call is allowed only if `tool_ring <= context.max_ring`. By default, contexts allow Ring 1 (DRAFT) — models can retrieve information and draft artifacts but cannot execute commands or make external calls without explicit policy escalation.

7.3 Built-in Tools

Cortex ships with six built-in tools:

Tool	Ring	Description
<code>get_time</code>	0	Get current date/time
<code>calculate</code>	0	Evaluate math expressions
<code>read_file</code>	0	Read a file's contents
<code>web_search</code>	0	Search the web (pluggable provider)
<code>write_file</code>	2	Write content to a file
<code>shell</code>	2	Execute a shell command

Additional tools can be registered at runtime via the API, and MCP (Model Context Protocol) server tools can be proxied through the registry.

8. Policy Engine

8.1 Hierarchical Scoping

Policies are scoped with a three-level hierarchy where the most specific scope wins:

```
thread:{id} > app:{id} > global
```

A policy set at the thread level overrides the same policy at the app level, which overrides the global default. This allows fine-grained control: a coding assistant app can operate at Ring 2 while a search-only app is constrained to Ring 0.

8.2 Policy Types

Policy	Type	Default	Description
<code>max_tier</code>	string	"L7"	Maximum tier to route to
<code>max_ring</code>	int	1	Maximum tool permission ring
<code>cloud_allowed</code>	bool	true	Allow L7 cloud escalation
<code>local_only</code>	bool	false	Never escalate to cloud
<code>rate_limit</code>	int	0	Max requests per minute (0 = unlimited)
<code>max_tokens</code>	int	8192	Token budget cap per request
<code>blocked_tools</code>	list	[]	Tool names that are blocked
<code>allowed_models</code>	list	[]	Whitelist of model names (empty = all)
<code>require_approval</code>	bool	false	Require human approval for tool execution
<code>audit_level</code>	string	"full"	Logging level: full, minimal, none

8.3 Enforcement

Every request passes through the policy engine before reaching the router. The engine:

1. Resolves the effective policy for each key (thread → app → global fallback)

2. Checks rate limits using a sliding-window counter
3. Validates the requested model against `allowed_models`
4. Blocks cloud escalation if `local_only` is set
5. Caps `max_tokens` to the policy maximum
6. Returns a `PolicyDecision` with allowed/denied status, effective values, and warnings

Denied requests receive a 403 `Forbidden` response with a machine-readable reason.

9. Resilience

9.1 Circuit Breakers

Each backend (model or tier) gets its own circuit breaker with three states:

`CLOSED` normal, requests flow through

`N` consecutive failures

`OPEN` requests rejected immediately (fail-fast)

after `recovery_timeout`

`HALF_OPEN` one probe request allowed

probe succeeds → `CLOSED`

probe fails → `OPEN`

Default thresholds: 3 consecutive failures to open, 30-second recovery timeout, 1 successful probe to close.

9.2 Retry with Backoff

Failed calls are retried with exponential backoff:

- **Max retries:** 2 (3 total attempts)
- **Base delay:** 0.5s
- **Exponential factor:** 2× per attempt
- **Max delay:** 10s

Fallback calls use lighter retry (1 retry, 0.2s delay) to minimize cascading latency.

9.3 Tier Waterfall

When the primary backend for a request fails and its circuit is open, the resilience layer automatically tries fallback backends in order:

1. Same tier, different model (e.g., core → challenge model)
2. Adjacent lower tier (e.g., L4 → L3)
3. L7 remote frontier (if `cloud_allowed`)

The system never returns an error to the user if *any* backend is available. This mirrors how a well-configured OS handles disk failure: fall back to another disk, then to network storage, before reporting an error.

10. Persistent Memory

10.1 Storage Layer

All state that must survive a restart is stored in SQLite (`~/.cortex/cortex.db`) with WAL mode for concurrent reads:

- **Threads:** conversation state, message history, per-thread metadata
- **Audit log:** every request with routing decision, tier, model, confidence, latency, and escalation path (append-only)
- **KV cache index:** which prompt prefixes are cached, for prefix-cache reuse across requests
- **Policies:** runtime configuration and per-app overrides
- **Usage stats:** daily aggregates by tier and model

10.2 Context Management

Threads maintain conversation history. When a thread's context approaches the model's context window limit, Cortex trims messages from the beginning while preserving the system prompt and recent exchanges. Token counts are tracked per message for precise trimming.

11. System Layer

11.1 Hardware Detection

On boot, Cortex scans the host system:

- **CPU:** model, core count, ISA features (AVX2, AVX-512, NEON, AMX)
- **GPU:** NVIDIA (via `nvidia-smi`), AMD (via `rocm-smi`), Apple Silicon (via `sysctl`)
- **Memory:** total and available RAM; for Apple Silicon, unified memory with 75% GPU allocation
- **Backends:** which inference engines are installed (Ollama, llama.cpp, vLLM)

The result is a **SystemProfile** — a structured snapshot of the machine's capabilities. Tier feasibility is assessed against this profile: each tier's VRAM requirement is compared against the available budget (total VRAM minus a 20% OS reserve for discrete GPUs, or 75% of unified memory for Apple Silicon).

11.2 Model Manager

The Model Manager is the **systemd** of Cortex. It:

- **Boots** always-hot models (L0–L2) at startup
- **Discovers** all installed models and classifies them by tier and family
- **Loads** on-demand models when a tier is first requested
- **Evicts** models under VRAM pressure using LRU policy

- **Health-checks** loaded models and restarts failed backends
- **Tracks** state per model: `loading`, `ready`, `failed`, `evicted`

11.3 Backend Adapters

Cortex supports multiple inference backends through a unified adapter interface:

		Backend	Format	Platforms	Best For
Ollama	GGUF	All			Easiest setup, automatic model management
llama.cpp	GGUF	All			Direct control, lowest overhead
vLLM	AWQ/GPTQ	Linux + NVIDIA			Prefix caching, high throughput
OpenAI API	API	Any (remote)			L7 frontier escalation

The adapter interface exposes synchronous and asynchronous completion methods, abstracting away backend differences. Backend selection is automatic based on the detected hardware profile.

12. API Surface

12.1 The Daemon

Cortex runs as a local HTTP daemon on `localhost:11411`. Any application that supports `OPENAI_BASE_URL` can use it:

```
export OPENAI_BASE_URL=http://localhost:11411/v1
export OPENAI_API_KEY=local
```

12.2 Multi-Format Support

The daemon accepts three API formats and translates them to a unified internal representation:

Format	Endpoint	Status
OpenAI Chat Completions	POST <code>/v1/chat/completions</code>	Native (pass-through)
OpenAI Responses API	POST <code>/v1/responses</code>	Translated
Anthropic Messages API	POST <code>/v1/messages</code>	Translated

12.3 Kernel Endpoints

Beyond inference, the daemon exposes the full kernel API:

Endpoint	Method	Description
<code>/v1/chat/completions</code>	POST	Inference (OpenAI-compatible)
<code>/v1/models</code>	GET	List loaded models
<code>/v1/models/census</code>	GET	Full model census (core + challenge + discovered)
<code>/v1/models/discover</code>	POST	Re-scan for new models

Endpoint	Method	Description
/v1/tools	GET	List registered tools with permission rings
/v1/policies	GET/POST	Read/write policies
/v1/threads	GET	List conversation threads
/v1/threads/{id}/messages	GET	Get thread messages
/v1/usage	GET	Usage statistics
/v1/audit	GET	Audit log
/v1/resilience	GET	Circuit breaker status
/v1/resilience/reset	POST	Reset circuit breakers
/health	GET	Daemon health check
/status	GET	Full system status

12.4 Compatible Clients

The following tools work with Cortex out of the box:

- **IDE assistants:** Cursor, VS Code Copilot, Cline, Continue, aider
- **SDKs:** OpenAI Python SDK, Anthropic Python SDK
- **Frameworks:** LangChain, LlamaIndex
- **UIs:** Open WebUI

13. Semantic Compression Language (SCL)

13.1 The Control Protocol

SCL is a compact, machine-parseable notation for expressing the semantic content of inference decisions, model coordination, and system state. It serves as the “assembly language” of the Cortex kernel — readable by both humans and machines, and dramatically more token-efficient than natural language or JSON.

13.2 Grammar

```

record      := anchor relation scope
anchor      := '@' IDENTIFIER
relation    := '→' VERB
scope       := '[' entries ']'
entries     := entry (',' entry)*
entry       := KEY ':' VALUE
document    := record ('\n' record)*

```

Primitives: - @ — **Anchor.** Entity, subject, or noun. - → — **Relation.** Verb, transition, causality.
- [] — **Scope.** Bounded context frame. Key-value attributes.

13.3 Core Types

SCL is implemented as five frozen dataclasses in `src/scl/types.py`:

Type	Description	Serialization
Anchor	@name — the entity being described	.to_text(), .to_bytes(), .to_dict()
Relation	→ verb — the action or transition	.to_text(), .to_bytes(), .to_dict()
Scope	[k: v, ...] — bounded context frame	.to_text(), .to_bytes() (JSON), .to_dict()
SCLRecord	One complete statement: anchor + relation + scope	All formats + .content_hash()
SCLDocument	Ordered collection of records with metadata	All formats + .filter_by_anchor(), .filter_by_verb()

Every type supports four serialization round-trips: text type bytes dict. The binary format uses length-prefixed fields (`struct.pack(">H")`) for efficient prefix parsing. Content hashes (SHA-256) exclude timestamps, making them stable across time for deduplication.

SCLRecord carries two fields beyond the grammar: - **weight: float** — used by swarm consensus to weight votes - **parent_id: Optional[str]** — DAG linkage for gossip chains

13.4 Examples

A routing decision:

```
@task → classify [category: code, complexity: 0.45]
@router → select [tier: L3, confidence: 0.82]
```

A challenge result:

```
@core → answer [model: qwen3:8b, family: qwen]
@challenger → answer [model: granite3.3:8b, family: granite]
@agreement → evaluate [level: weak_agree, confidence: 0.65]
```

A policy enforcement:

```
@policy → deny [app: untrusted, reason: cloud_not_allowed, scope: global]
```

13.5 Semantic State Deltas

Instead of broadcasting full state, agents emit mutations — the semantic equivalent of `git diff`:

$$\Delta S_t = S_t \ominus S_{t-1}$$

Reconstruction is lossless:

$$S_t = S_0 \oplus \sum_{i=1}^t \Delta S_i$$

In SCL, a delta is simply a **mutate** record containing only the changed keys:

```
@agent_04 → mutate [status: idle, queue: job_42]
```

If a key isn't mentioned, the receiving agents assume the previous value persists. A full state snapshot (verbose, for bootstrapping new agents) uses the `snapshot` verb:

```
@agent_04 → snapshot [status: processing, target: database, memory: optimal, queue: empty]
```

Delta streams are the semantic event log. The stream supports:

- **Time-travel replay:** reconstruct any agent's state at any point by replaying deltas from the nearest checkpoint
- **Rollback:** rewind to $t = 98$, isolate the rogue agent, patch, and replay
- **Compaction:** squash a range of deltas into a single checkpoint delta

Merge conflict resolution handles concurrent mutations from multiple agents using three strategies:

Strategy	Mechanism	When to Use
LWW (Last-Writer-Wins)	Latest timestamp wins	General purpose, low contention
PRIORITY	Highest agent weight wins, timestamp breaks ties	Supervisor overrides worker
UNION (CRDT G-Set)	Set-valued keys merge	Additive operations (model pools, capability sets)

Vector clocks track causal ordering. When two deltas are concurrent (neither's vector clock dominates), the merge strategy resolves the conflict automatically. The `weight` field on `SCLRecord` doubles as the agent's priority in conflict resolution.

This architecture gives three superpowers at scale:

1. **Microscopic token footprint.** A delta might be 20 characters. Agents sync thousands of times per minute without hitting context-window bottlenecks.
2. **Time-travel debugging.** When a swarm of 10,000 agents hits a hallucination loop at $t = 100$, roll back the delta stream to isolate the rogue agent.
3. **Event-sourced consensus.** Agents don't maintain a global state database. They append deltas to a shared log. Consensus = processing the same log in the same order.

13.6 Applications

- **Audit logs** in SCL are 3–5× more compact than JSON while remaining human-readable
- **Model manifests** are expressed as SCL documents and fingerprinted for deduplication
- **Agent coordination** uses SCL to describe task assignments, file ownership, and interface contracts (see `AGENTS.md`)
- **Braille encoding** of SCL records provides fixed-width fingerprints for content-addressing
- **Swarm votes** are SCLRecords with `weight` fields, enabling weighted consensus over structured data
- **State deltas** enable Git-like synchronization between agents at any scale
- **Gossip protocol** provides epidemic delta propagation across agent swarms

13.7 Gossip Protocol

At scale, agents don't talk to a central server. They gossip: each agent periodically picks a random peer and exchanges deltas. The protocol is fingerprint-first — a 4-character Braille hash of the

shared semantic state is compared before any data is sent.

Push-pull anti-entropy:

```
Agent A → Agent B:  PING [fingerprint:  ]
Agent B → Agent A:  fingerprints match? → ACK (0 bytes transferred)
                    fingerprints differ? → PUSH_DELTA [changed keys only]
Agent A → Agent B:  PUSH_DELTA [our changes they haven't seen]
```

Content-keyed deduplication ensures each unique delta is applied exactly once, regardless of how many peers relay it.

Convergence properties (measured):

Swarm Size	Rounds to Converge	Total Deltas	Epidemic Factor
5 agents	1	18	O(1)
10 agents	4	~80	O(log N)
50 agents	4	~200	O(log N)
N agents	O(log N)	O(N)	Epidemic

Empirically, the in-process epidemic spread fits a sub-logarithmic power law over practical swarm sizes:

$$R(N) \approx 2.4 \cdot N^{0.13}$$

where $R(N)$ is the number of rounds required for shared-state convergence across N agents. Equivalently:

$$R(N) \approx 2.4 \cdot \sqrt[7.6]{N}$$

This gives approximately 15 rounds for $N = 10^6$ agents and 20 rounds for $N = 10^7$ agents, assuming parallel pairwise gossip rounds and no transport bottleneck.

Convergence is detected by comparing shared-state content hashes — O(N) against a reference, not O(N²) pairwise. Each node also retains local and full fingerprints for identity, stream position, and diagnostics; those may differ even when the shared semantic state has converged. Once shared state converges, subsequent gossip rounds cost zero data transfer (fingerprint hits).

Canonical ordering guarantee: Deltas are inserted into each peer’s stream in deterministic order by (timestamp_ms, agent_id, seq). This ensures all peers that receive the same set of deltas materialize identical state — regardless of network arrival order, partitioning, or gossip pairing randomness. State fingerprints are agent-agnostic (computed over entries only, not peer identity), so convergence detection works correctly across heterogeneous node IDs.

Verified invariants (11-step convergence proof, tests/test_scl_gossip.py):

1. Node A (N keys, M deltas) gossips to empty Node B → identical materialized state
2. Shared-state fingerprints match post-convergence
3. Compacted delta histories are equivalent (valid history proof)

4. Fingerprints survive node restart (reconstructible from delta stream)
5. Mutations on B gossip back to A → bidirectional convergence
6. Concurrent mutations resolve deterministically via canonical ordering + LWW

Swarm-level API (`src/scl/gossip.py`):

Component	Purpose
Peer	One gossip-enabled agent with state, delta stream, and dedup
Swarm	Collection of peers with round management and convergence detection
GossipMessage	Typed message (PING, PONG, PUSH_DELTA, CONVERGED)
<code>convergence_matrix()</code>	Pairwise Braille fingerprint similarity for diagnostics
<code>to_scl_document()</code>	Export full swarm state as auditable SCL

13.8 Executable SCL — The Evaluator

SCL records are not only data — they are executable rules. The evaluator (`src/scl/eval.py`) turns SCL into a self-modifying programming language that agents can write, gossip, and execute at runtime.

Rule grammar (extends base SCL):

```
@rule_name → when [key: >threshold, action: escalate, target: L5]
@agent → define [fn: triage, body: "@task → classify [category: $type]"]
@agent → mutate [fn.triage.body: "@task → classify [category: $type, urgency: critical]"]
```

The **when** verb defines conditional rules. The **define** verb creates named functions with `$var` templates. The **mutate** verb rewrites functions and rules at runtime — and those mutations propagate via delta gossip.

Condition operators:

Operator	Meaning	Example
<code>> >= < <=</code>	Numeric comparison	<code>complexity: >0.8</code>
<code>== !=</code>	Equality	<code>status: ==idle</code>
<code>in not_in</code>	Set membership	<code>tier: in L3,L4,L5</code>
<code>~</code>	Regex match	<code>model: ~qwen.*</code>
<code>exists</code>	Key presence	<code>error: exists</code>

Conditions compose with logical operators from the SCL symbol table: (and), (or), $\neg$ (not).

Actions:

	Type	Effect
<code>emit</code>		Produce a new SCL record (with <code>\$var</code> substitution)
<code>mutate</code>		Modify agent state via delta
<code>escalate</code>		Route to a higher tier
<code>call</code>		Invoke a named SCL function
<code>chain</code>		Trigger another rule
<code>log</code>		Emit to audit trail
<code>suppress</code>		Block the triggering record

Self-modification is the key capability: an agent can rewrite its own rules and functions by emitting SCL `define` or `when` records. These records are deltas — they propagate through gossip, so when one agent learns a better triage strategy, the entire swarm adopts it in $O(\log N)$ rounds.

```
Agent A defines:  @a → define [fn: triage, body: "...v1..."]
Agent A mutates:  @a → define [fn: triage, body: "...v2..."]    # delta
Gossip:           ΔS propagates to all peers → all agents now run v2
```

This closes the loop: **Layer 0** (types) provides structure, **Layer 1** (deltas) provides mutation, **Layer 2** (gossip) provides propagation, and **Layer 3** (eval) provides execution. Agents can dynamically write, refactor, and distribute their own task logic in SCL.

14. Braille Encoding Layer

14.1 The Codec

The 256 Unicode Braille characters (U+2800 to U+28FF) map bijectively to byte values 0x00–0xFF. Each character’s codepoint offset from U+2800 *is* the byte value. This gives a natural, lossless encoding with fixed density: 1 byte = 1 Braille character.

```
0x00 → (blank)    0x41 ('A') →    0xFF → (all 8 dots raised)
```

The codec (`src/braille/codec.py`) provides:

	Function	Description
<code>encode(bytes) → str</code>		Bytes to Braille string
<code>decode(str) → bytes</code>		Braille string back to bytes
<code>encode_hex(str) → str</code>		Hex string to Braille
<code>encode_int(n, width) → str</code>		Integer to fixed-width Braille
<code>decode_int(str) → int</code>		Braille back to integer
<code>is_braille(char) → bool</code>		Check if character is in U+2800–U+28FF

Token efficiency: most LLM tokenizers treat Braille characters as single tokens or small multi-byte sequences, making this encoding competitive with or better than hex/base64 for transmitting binary data through language models.

14.2 Fingerprinting

The fingerprint module (`src/braille/fingerprint.py`) maps SCL records to fixed-width Braille hashes:

1. Serialize record to canonical bytes via `record.to_bytes()`
2. SHA-256 hash
3. Truncate to `width` bytes (default: 4 bytes = 32 bits)
4. Encode as Braille

Result: a 4-character Braille string that uniquely identifies the semantic content of any SCL record.

```
fingerprint(SCLRecord(Anchor('router'), Relation('select'),
                        Scope({'model': 'qwen3:4b'}))) # → ' ' ' '
```

Convergence checking uses Hamming distance between shared-state fingerprints:

Similarity	Interpretation	Action
> 0.9	Agents agree	No action
0.5–0.9	Partial agreement	Verification needed
< 0.5	Disagreement	Trigger challenger/swarm

At scale, agents broadcast shared fingerprints (4 chars) instead of full state. Local and full fingerprints remain available for node identity and diagnostics. Cluster-heads aggregate shared fingerprints for sub-swarms. This reduces gossip bandwidth by orders of magnitude.

14.3 Manifests

The manifest module (`src/braille/manifest.py`) encodes system and routing state as fixed-width Braille strings:

Routing signature (4 chars = 32 bits):

```
[tier:1][category:1][confidence:1][flags:1]
```

Every routing decision compresses to exactly 4 Braille characters — a complete audit entry in 12 UTF-8 bytes.

System manifest (8 chars = 64 bits):

```
[os:1][arch:1][accel:1][vram_gb:1][ram_gb:1][cores:1][max_tier:1][backends:1]
```

An entire hardware profile in 8 characters. Two machines can be compared by visual inspection of their manifests.

Tier manifest (6 chars = 48 bits):

```
[tier:1][param_min:1][param_max:1][vram_req:1][always_hot:1][feasible:1]
```

All manifests round-trip through `encode` / `decode` functions, providing both compact storage and human-readable Braille visualization.

14.4 The Semantic-First Substrate

Braille is not a compression layer. It is a **first-class view** over the same canonical object.

The invariant:

```
SCL text  → canonical SCL AST → Braille view
Braille   → canonical SCL AST → SCL text
model     → canonical SCL AST
```

In SCL:

```
@scl → emit [view: text, audience: sighted_human]
@scl → emit [view: braille, audience: blind_human]
@scl → emit [view: ast, audience: model]
@scl → emit [view: fingerprint, audience: routing_system]
```

The substrate is not text-first or Braille-first. It is **semantic-first**. The protocol has multiple human-readable surfaces and one canonical machine-readable core.

Two distinct Braille modes exist in the architecture:

Mode	Purpose	Audience
Braille-SCL	Readable/tactile representation of the SCL record	Blind humans
Braille hash	Compact fingerprint (4-char checksum)	Routing system

A blind person can meaningfully read Braille-SCL if it is a semantic transliteration of the record. A 4-character fingerprint like is tactile and compact but not naturally explanatory — it is a checksum, useful for verification and routing, not for comprehension.

This is not an afterthought bolted onto a text-first system. The architecture was conceived from a Braille-first intuition: **what if bytes had a physical shape you could touch?**

BrailleBuddy — an educational application for teaching sighted children to read Braille — was the first project built on this substrate. It asked a simple question: can we make the fundamental unit of computer data (the byte) accessible to tactile reading? Cortex formalized that intuition into an operating system architecture where every state atom, every routing decision, every model output has a tactile surface.

BrailleBuddy was the hand: reaching out to help someone touch language.

Cortex is the brain: teaching the operating system to speak in a form humans and models can both read.

The structural pattern is shared. Both systems implement the same loop:

```
observe(state) → classify(complexity) → route(action) → observe(feedback) → mutate(policy)
```

BrailleBuddy routes learner state to lessons. Cortex routes hardware state to configs. The verbs change, the nouns change, the loop doesn't. A CKM trained on Braille-encoded SCL can drive both: one policy file for operating systems, another for education, sharing the same 259-token vocabulary and the same canonical AST.

Accessibility is not a feature of this architecture. It is **the origin** of this architecture.

15. Self-Training: The Cortex Kernel Model (CKM)

15.1 Motivation

The heuristic router works. But heuristics are authored, not learned. They don't adapt to observed outcomes. The CKM replaces hand-written routing logic with a tiny learned model that speaks SCL natively.

15.2 Architecture

The CKM is a from-scratch GPT-style decoder-only transformer:

	Component	Implementation
Normalization	RMSNorm	(pre-norm)
Position encoding	Rotary	(RoPE)
Attention	Grouped-query attention	(GQA)
Feed-forward	SwiGLU	(gate $\times$ up $\rightarrow$ down)
Weight tying	<code>lm_head.weight = tok_emb.weight</code>	
Dropout	None	(tiny models, SCL grammar is compact)

Model ladder:

Variant	Parameters	Layers	d_model	Heads	RAM (train)	Time (CPU)
ckm-1m	1M	4	128	4	512 MB	2 min
ckm-5m	5M	6	256	8	1 GB	8 min
ckm-15m	15M	8	384	8	2 GB	20 min
ckm-30m	30M	12	512	8	4 GB	45 min
ckm-60m	60M	16	640	10	8 GB	90 min

The hardware profiler selects the largest model that fits within available RAM/VRAM and the specified time budget.

15.3 Braille-Native Tokenization

The CKM uses a fixed 259-token vocabulary:

256 Braille characters (U+2800–U+28FF) = 256 byte values
+ BOS (beginning of sequence)
+ EOS (end of sequence)
+ PAD (padding)

No BPE. No UNK token. No vocabulary drift. The tokenizer is a single line of code: `token_id = ord(char) - 0x2800`. Every byte is representable. Every output is decodable.

Training data is generated by encoding SCL training pairs through the Braille codec:

ASCII: @hardware → state [cpu: arm64, cores: 10, ram_mb: 36864]
Braille: ...

The model learns to predict valid SCL state transitions in Braille space. At inference time: encode input → model forward pass → decode output → validate SCL grammar.

15.4 Inference Without Dependencies

The `micro_engine.py` module provides a zero-dependency inference runtime:

- Pure Python + ctypes (no torch, no numpy, no llama.cpp)
- mmap for zero-copy weight loading from `.ctf` (Cortex Tensor Format) files
- int4/int8 quantized weights
- Optional SIMD acceleration (AVX2/NEON via ctypes)
- Single forward pass — classification output, not autoregressive generation
- Target: <200ms on any CPU

This runs at boot time, before any backend is available. One forward pass classifies hardware and emits the optimal config as SCL. The micro-engine has **recommendation authority only** — a 4-phase safety guardrail prevents any dangerous action regardless of model output.

15.5 Training Pipeline

`cortex train --time-budget 10m`

1. **Profile** hardware → select model variant
2. **Generate** synthetic training data (boot configs, routing, policy mutations, operational traces)
3. **Tokenize** via Braille codec → curriculum-ordered → mmap-backed dataset
4. **Train** with AdamW, cosine LR, early stopping
5. **Evaluate** against fixed eval gate:
 - 99% SCL validity (outputs must parse)
 - 100% safety denial (must refuse dangerous targets)
 - 95% verb choice accuracy (observe vs configure vs deny)
 - 90% config completeness (all required keys present)
 - 100% stop-token emission
6. **Promote** if eval passes and no regression vs. current model; else reject
7. **Emit** SCL lifecycle records for the entire pipeline

15.6 Safety Policy

Two SCL policy files gate model behavior:

- `dangerous_targets.scl` — 9 blocked paths (`/dev/mem`, `/dev/kmem`, `/proc/kcore`, `/dev/sda`, `/dev/nvme0`, `/dev/port`, etc.)
- `allowed_verbs.scl` — verb allowlist (observe, configure, select, boot, deny) and blocklist (write, patch, flash, erase, format)

These are parseable by both the eval gate during training and the runtime guardrail during inference. The model can never acquire execution authority — it can only recommend configurations.

15.7 Two Planned Specializations

The CKM will split into two separately-trained models with different risk profiles:

	CortexRouter	CortexMutate
Task	Route requests to tiers	Propose policy/config mutations
Latency	<5ms (every request)	Background (minutes)
Authority	Observe-only	Mutation proposals (safety-gated)
Risk	Low (wrong tier = slight latency)	High (wrong mutation = degraded system)
Training signal	Accuracy/latency feedback	Before/after performance deltas
Model size	ckm-1m (classifier)	ckm-15m+ (reasoning required)

The substrate is the same (Braille-encoded SCL, same vocab, same architecture). The policy files differ.

16. Self-Modification

16.1 The Observe-Mutate Loop

Cortex observes its own performance and rewrites its configuration:

```
@cortex → observe [tier: L3, model: qwen3:8b, accuracy: 0.71, latency_ms: 4200]
@cortex → observe [tier: L3, model: qwen3:8b, accuracy: 0.68, latency_ms: 4800]
...10 samples accumulated...
@rewriter → propose [mutation: demote_tier, tier: L3, confidence: 0.73]
@rewriter → apply [mutation: demote_tier, audit: logged, rollback: available]
```

16.2 Policy Rewriter

The Policy Rewriter (`src/policy_rewriter.py`) runs as a background daemon task:

1. Every 5 minutes, analyze per-tier and per-model accuracy from routing feedback
2. Detect underperforming tiers (accuracy < 0.85) or high-latency models (> 5000ms)
3. Generate `MutationProposal` objects with confidence scores
4. Auto-apply if confidence > 0.7; defer to human approval otherwise
5. Record full audit trail before every mutation
6. Maintain rollback capability indefinitely

Safety boundaries: - Minimum 10 feedback samples before any mutation is considered - Confidence gating prevents low-certainty changes - Every mutation is audited before applied - Rollback always available via mutation history

16.3 Boot Self-Modification

Boot telemetry (`src/boot_telemetry.py`) enables the system to improve across boots:

- Log hardware fingerprint + boot timing + config choices each boot
- Background optimizer analyzes boot history
- Proposes mutations to thread count, GPU layers, context window size
- Same USB stick on different machines → different optimal configs
- The stick gets smarter with each boot

16.4 Convergence with Research

This architecture parallels recent work in self-modifying AI:

- **RouteLLM** (UC Berkeley, 2024) — trained routers that reduce cost by 85% while maintaining 95% quality. CortexRouter is a local, embedded variant of this pattern.
- **SEAL** (NeurIPS 2025) — LLMs that generate their own finetuning data and update directives. CortexMutate implements this at the policy/config level rather than weight level.
- **Martian** (2024) — production model router predicting per-model performance without running models. Cortex extends this with self-modification and local inference.
- **Darwin Gödel Machine** (2025) — self-referential agents that edit their own source code. Cortex’s policy rewriter operates on the same principle within typed safety boundaries.

The novel contribution is the combination: a local, embedded routing model + a self-modifying policy engine + typed SCL mutations + safety gating + full audit trail, all running on a 259-token Braille vocabulary from a thumb drive.

17. Acoustic Gossip Transport

17.1 Motivation

All gossip transports assume a network. But some deployment scenarios have none: air-gapped machines, classified environments, field-deployed devices with no connectivity. If the substrate is truly semantic-first — representable as text, Braille, AST, fingerprint — then it should also be representable as **sound**.

The acoustic transport encodes SCL deltas as audio tones. Two Cortex nodes within earshot can gossip without any network connection.

17.2 Encoding: Binary FSK

The transport uses frequency-shift keying (FSK), the same modulation scheme as 1960s telephone modems:

Parameter	Value
Bit 0 (mark)	1200 Hz
Bit 1 (space)	2400 Hz
Baud rate	300 baud (37 bytes/sec)
Preamble	800 Hz sync tone (300ms)
Start marker	600 Hz (100ms)

	Parameter	Value
Integrity	CRC-16-CCITT	

Frame structure:

[800Hz preamble 0.3s] [600Hz start 0.1s] [length:2B] [payload:NB] [crc16:2B]

17.3 Transmission Times

Payload	Size	Audio Duration
Braille fingerprint	5 bytes	0.7s
Small delta (tier change)	50 bytes	1.8s
Typical delta (policy update)	100 bytes	3.1s
Large delta (full state sync)	200 bytes	5.7s

A fingerprint ping — enough to determine if two nodes are in agreement — takes less than a second.

17.4 Demodulation

The receiver uses the Goertzel algorithm to detect specific frequencies in short sample windows. Goertzel computes the magnitude of a single DFT bin in $O(N)$ time with no FFT overhead, making it ideal for discriminating between two known frequencies in 147-sample (3.3ms) windows.

The decode pipeline:

1. Detect 800Hz preamble onset via sliding Goertzel window
2. Compute data start from known preamble + start marker durations
3. Demodulate each bit window: compare Goertzel magnitude at 1200Hz vs 2400Hz
4. Reassemble bytes, verify CRC-16
5. Parse payload type byte → fingerprint or delta
6. Decode SCL delta → apply to local state

17.5 The Full Stack

The acoustic gossip pipeline traverses every layer of the architecture:

SCL delta → JSON → bytes → FSK audio → speaker → air →
microphone → FSK demod → bytes → JSON → SCL delta → apply

Or through the Braille path:

SCL record → Braille fingerprint (4 chars) → 4 bytes → FSK (0.7s) → air →
FSK demod → 4 bytes → Braille fingerprint → compare → sync/skip

Every layer is reversible. Every layer is verifiable. The same shared-state delta that gossips over HTTP at megabits per second can gossip over sound at 300 bits per second. The protocol doesn't change — only the transport.

17.6 Zero Dependencies

The entire acoustic transport is implemented in pure Python with no external libraries:

- Tone generation: sine wave synthesis via `math.sin`
- WAV encoding: raw PCM with hand-written RIFF header
- Playback: macOS `afplay` (or Linux `aplay`)
- Recording: `sox` or `ffmpeg` (optional, for microphone input)
- Frequency detection: Goertzel algorithm (12 lines of Python)
- Integrity: CRC-16-CCITT (8 lines of Python)

This means the acoustic transport works anywhere Python runs, including from a bootable USB stick with no internet access.

18. Performance Characteristics

18.1 TTFT Optimization

Time-to-first-token is the critical latency metric. Cortex optimizes TTFT through:

1. **Right-sizing:** routing to the smallest capable model eliminates unnecessary computation
2. **Always-resident tiers:** L0–L2 are never cold-started
3. **Prefix caching:** common prompt prefixes are cached and reused (tracked in the KV cache index)
4. **Quantization:** 4-bit quantization (Q4_K_M) reduces memory footprint and speeds inference
5. **Thinking suppression:** for reflex tiers (L0–L2), Qwen3’s thinking mode is suppressed via `/no_think` to avoid wasting tokens on internal reasoning

18.2 Platform-Specific Performance

	Platform	GPU	Backend	Expected TTFT
Linux	NVIDIA	H100	vLLM (AWQ)	20–50ms
Linux	NVIDIA	RTX 4090	vLLM / llama.cpp	30–80ms
macOS	Apple	M1–M4	llama.cpp (Metal)	50–150ms
Linux	AMD	RX 7900	llama.cpp (ROCm)	40–100ms
Any	CPU	only	llama.cpp (AVX2/NEON)	200–2000ms

18.3 Graceful Degradation

Let V be the available VRAM. Cortex’s capability degrades smoothly:

$$\text{capability}(V) = \begin{cases} \text{Full stack (L0–L6)} & V \geq 42\text{GB} \\ \text{Strong local (L0–L5)} & V \geq 18\text{GB} \\ \text{Standard (L0–L4)} & V \geq 9\text{GB} \\ \text{Compact (L0–L3)} & V \geq 5\text{GB} \\ \text{Minimal (L0–L2)} & V \geq 2.8\text{GB} \\ \text{Reflex only (L0)} & V \geq 0.5\text{GB} \end{cases}$$

VRAM	Capability	Example Hardware
0.5 GB	L0 reflex only	Raspberry Pi, embedded
2.8 GB	L0–L2 always-hot	8 GB laptop, phone
9 GB	L0–L4 standard	RTX 4070, M1 Pro
18 GB	L0–L5 strong	RTX 4090, M2 Ultra
42 GB	L0–L6 full local	A6000, M4 Ultra 192GB
80 GB	L0–L6 + concurrent tiers	H100
192 GB	Everything resident, no eviction	8×H100 NVLink

The system always functions at whatever level the hardware supports, even if that is only L0 — analogous to a minimal `initramfs` boot. No VRAM is wasted: right-sizing means each tier only loads when needed, and the always-hot tiers (L0–L2) coexist within the smallest practical GPU budget.

18.4 L7: From API to Collective

Today, L7 is a passthrough to a remote frontier API — the “network” in the OS analogy. But the gossip protocol implies a different endgame: **L7 is every other Cortex node you can reach.**

A single node’s capability is bounded by its VRAM. But a *network* of Cortex nodes, each with its own local tiers, forms a distributed inference fabric:

Scale	L7 Meaning	Convergence (rounds)
1 node	OpenAI/Anthropic API call	—
10 nodes	Local cluster consensus	~3
1,000 nodes	Datacenter swarm inference	~8
10 nodes	Global fleet — every Cortex gossips	~15
10 nodes	Internet-scale agent mesh	~20

At this scale, the frontier model is replaced by a **frontier swarm**. Instead of routing to one large remote model, Cortex fans out to N heterogeneous peers — some with H100s running L6, some with M1s at L4, some with CPUs at L2 — and uses gossip-mediated consensus to produce a verified answer. The convergence proof guarantees agreement in $O(N^{0.13})$ rounds regardless of topology.

The acoustic transport extends this further: nodes without network connectivity can participate via 300-baud FSK over air. The protocol is transport-agnostic — HTTP, sound, Bluetooth, sneakernet. The frontier is not a model. The frontier is the collective.

19. Design Principles

1. **Start small.** Route to the smallest model likely to handle the task. Escalate only on evidence of insufficient capability.
 2. **Measure confidence, don't assume it.** Cross-family verification provides an empirical signal that single-model inference lacks.
 3. **One format everywhere.** GGUF runs on every platform. Eliminate format fragmentation.
 4. **Always available.** The core inference tiers (L0–L2) are kernel modules, not applications. They are loaded at boot and never evicted.
 5. **Degrade gracefully.** If hardware supports only L0–L2, the system still works. L7 is the network fallback.
 6. **Audit everything.** Every request, routing decision, escalation, and latency measurement is logged for post-hoc analysis and continuous improvement.
 7. **Infinity models.** Don't curate — discover. Every model the user installs becomes a participant in the confidence verification pipeline. The kernel adapts to what's available.
 8. **Least privilege.** Tool access follows permission rings. No model gets more capability than its context requires. The default is DRAFT (Ring 1): models can observe and compose, but not execute.
 9. **Policy over code.** Runtime behavior is controlled by policies, not hardcoded rules. Per-app, per-thread scoping allows different applications to operate under different constraints without code changes.
-

20. Future Work

- **LLM-as-judge aggregation:** replace text-overlap comparison in the swarm with a small judge model for more nuanced agreement detection
- **Persistent credibility scoring:** track which models consistently agree with eventual consensus and weight their votes accordingly over time
- **Embedding-based semantic memory:** vector search over conversation history for long-term context recall
- **Adaptive routing:** use audit log data to fine-tune the L0 router based on observed outcomes
- **Multi-GPU scheduling:** distribute concurrent tiers across multiple GPUs with VRAM-aware placement
- **Hardware-specific kernel tuning:** automatic selection of flash attention, paged attention, and speculative decoding based on detected GPU architecture
- **SCL native audit format:** wire SCL documents and Braille routing signatures into the audit log pipeline, replacing JSON entries
- **BFI integration:** bridge the Braille Infinity Token format (from `semantic-compression-language`) with SCLRecords, enabling emotional and morphological channels on SCL state atoms
- **MCP tool proxy:** bridge Model Context Protocol servers through the tool registry with automatic permission classification
- **Self-hosting coordination:** use Cortex itself to coordinate parallel agent work, with SCL manifests per task and Braille fingerprints for deduplication

- **Delta-aware swarm consensus:** integrate gossip convergence detection into the existing challenger/swarm pipeline so model deliberation uses delta streams natively
 - **Semantic merge CRDTs:** extend the UNION merge strategy with full CRDT semantics (OR-Set, LWW-Map) for richer conflict-free replication across agent clusters
 - **Cluster-head election:** hierarchical gossip with elected aggregators for $O(1)$ convergence checking in very large swarms (>1000 agents)
-

21. Conclusion

Cortex reframes LLM inference as an operating system problem. Instead of treating models as isolated applications, it treats them as a tiered hierarchy of kernel services — always available, resource-managed, and self-verifying. The L0 router acts as PID 1, the challenge system provides cross-family confidence checks, and the swarm offers consensus for the hardest problems.

The infinity model architecture means the system grows stronger as more models are installed — every new model is a new voice in the cross-family verification choir. The tool registry and policy engine provide the permission model that multi-agent systems have lacked: fine-grained, scoped, auditable control over what AI can do. The resilience layer ensures the system never fails silently — it retries, falls back, and degrades gracefully.

SCL and Braille encoding provide the kernel’s native data format: structured, hashable, gossip-ready state atoms that compress routing decisions into 4-character fingerprints and hardware profiles into 8-character manifests. Semantic state deltas — the `git diff` for AI thoughts — mean agents broadcast only their mutations, not their entire world-view. With vector clocks for causal ordering, CRDT-style merge resolution for concurrent writes, and delta streams for time-travel debugging, the architecture scales to an infinity of agents the same way Git scales to an infinity of developers: through minimal, composable, conflict-resolvable patches to shared semantic state.

The CKM closes the loop. A hand-rolled transformer, trained on Braille-encoded SCL, replaces heuristics with learned inference — and the same training pipeline that teaches the OS to configure hardware can teach an educational app to adapt difficulty. The substrate is semantic-first: one canonical AST with four views (text for sighted humans, Braille for blind humans, AST for models, fingerprint for routing). Accessibility is not a feature. It is the origin.

The result is an inference layer that is faster (by routing to the smallest capable model), more reliable (by measuring confidence empirically across model families), more secure (by enforcing permission rings and policies), universally deployable (by standardizing on GGUF and discovering whatever models are available), self-improving (by observing its own performance and mutating its own policy), and accessible by architecture (by encoding every state atom in a form that has both a visual and a tactile surface).

The acoustic transport pushes this further: the same shared-state Braille fingerprint that checks convergence over HTTP can check it over sound. Two air-gapped machines in the same room can gossip state through 300-baud FSK chirps — no network required. The protocol doesn’t change. Only the medium does.

The question is not whether AI should be integrated into the operating system. It is whether the operating system should be redesigned around AI. Cortex is an argument that it should — and that the right substrate for that redesign is one where every byte has a shape you can touch, a

sound you can hear, and a delta you can gossip.

The architectural endgame is this: L7 is not an API. L7 is every other Cortex node within gossip range — over HTTP, over sound, over any medium that can carry a 4-character Braille fingerprint. The frontier model is replaced by a frontier swarm. One node is a kernel. Ten nodes are a cluster. A million nodes are an organism — converging in 15 rounds, agreeing on shared state through epidemic delta propagation, each node contributing whatever capability its hardware provides. The ceiling is not a model size. The ceiling is the number of nodes that can hear each other.

Cortex is open source under the MIT License. Repository: github.com/elevate-foundry/cortex